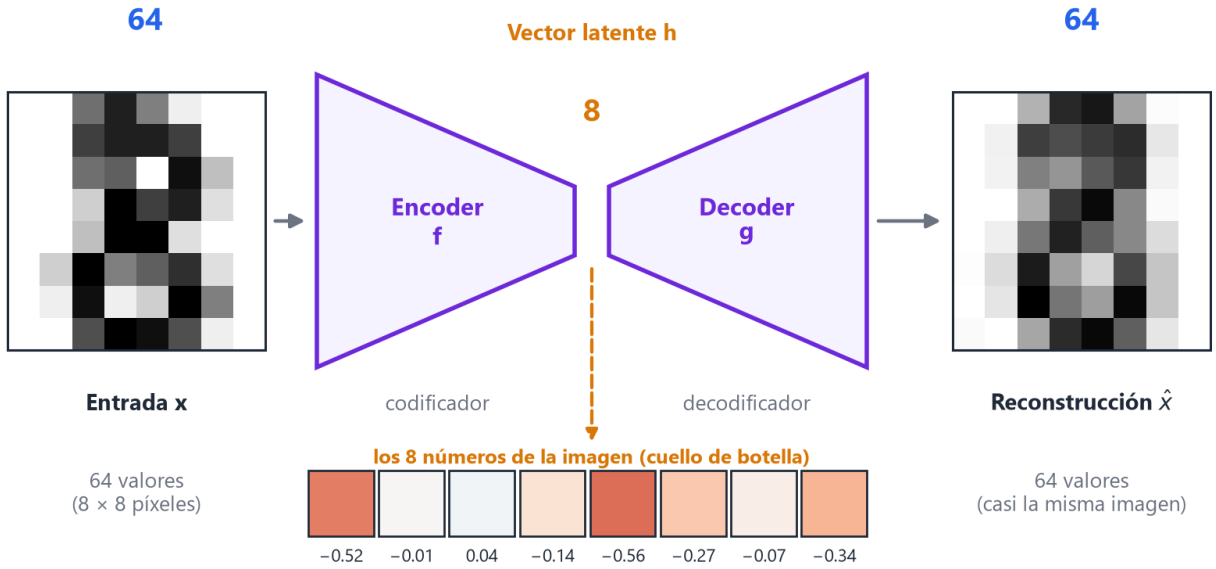
Clase 46 · Deep Learning, autoencoders y representaciones

Profesor Eloy Alvarado · la red ya no clasifica: aprende a resumir una imagen en 8 números y a volver a armarla, con los dígitos de las Clases 44 y 45

La gran idea de la clase

Un autoencoder es una red neuronal que se entrena para reproducir su propia entrada pasando por un resumen pequeño. El **encoder** convierte las 64 intensidades de una imagen en un **vector latente** de 8 números; el **decoder** usa solo esos 8 números para volver a estimar las 64 intensidades. La referencia con la que se compara es la **misma entrada**, así que no hacen falta etiquetas. El error se mide con el **MSE por píxel** y se reparte hacia las dos mitades con la misma retropropagación de la Clase 45. Como el resumen es más pequeño que la imagen, **se pierde información**: lo que importa es qué se conserva y si eso sirve para el uso previsto (buscar parecidos, limpiar ruido, priorizar revisiones). Un error de reconstrucción alto puede servir de **alerta**, pero es un puntaje, no una probabilidad, y exige una validación específica.

Autoencoder: la entrada se resume en 8 números y se vuelve a armar



Ambas mitades aprenden juntas con un solo error: qué tan distinta quedó $\hat{x}$ de x. La etiqueta del dígito no interviene.

Lo que necesitas ANTES

- **Red neuronal** (Clase 45): neurona, capa, activación, pérdida, gradiente, Adam, lotes y épocas
- **PCA** (Clase 32): proyectar a pocas direcciones y reconstruir
- **Validación y prueba** (Clases 30 y 38): elegir con una, evaluar al final con otra
- **Detección de anomalías** (Clase 41): un puntaje y un umbral

Ruta de esta guía

- Representación y autoencoder → reconstruir frente a clasificar → qué se pierde
- El error de reconstrucción (MSE) a mano → referencias: imagen media y PCA con 8 componentes
- Profundidad, parámetros y entrenamiento → mirar el código y usarlo
- El error como alerta de anomalías y sus límites

Cómo leer los números de este material

La grabación disponible termina en el recreo (1:14:23): **cubre las diapositivas 1 a 14**. Las diapositivas 15 a 53 (PCA, entrenamiento, resultados, anomalías, código en R) no están en el video; aquí se explican a partir de las diapositivas y del programa del profesor. Las cifras de las diapositivas son la corrida del profesor: se ejecutó **su mismo programa** (`autoencoders.py`, scikit-learn 1.9.1, misma semilla) y **las 18 cifras comparadas coinciden exactamente** (tabla en la última página). Lo que se rotula «nuestra reproducción» o «ejemplo propio» es cálculo propio, no de la clase.

1 Qué es una representación y qué es un autoencoder

Un autoencoder resume la entrada en un vector latente y luego intenta reconstruirla con solo ese vector. Se escribe h = f(x) para el encoder y $\hat{x} = g(h)$ para el decoder.

1. **Deep learning.** Redes que componen varias capas de transformación. No hay un número de capas desde el cual una red «es profunda»: más capas permiten más composiciones, pero su utilidad debe evaluarse (diapositiva 2).
2. **Representación.** Una forma de describir un objeto con números. La imagen se describe con 64 intensidades; el vector latente la describe con solo 8. Cada capa intermedia de una red aprende una representación.
3. **Encoder y decoder.** El **encoder** f calcula h a partir de x; el **decoder** g calcula $\hat{x}$ a partir de h. Son dos funciones con parámetros propios (θ para f y ψ para g, que en la Clase 45 se llamaban W_1 y W_2) y se ajustan juntas.
4. **Vector latente.** Es la salida de la capa central: se llama «latente» porque está *dentro* de la red. Aquí tiene 8 coordenadas, una decisión que se toma antes de entrenar (un hiperparámetro).

En palabras simples

Imagina que tienes que describir una foto de 64 cuadraditos a un amigo, pero **solo puedes dictarle 8 números**. Tu amigo (el decoder) dibuja la foto usando solo esos 8 números. El juego consiste en elegir tan bien los 8 números (el encoder) y la forma de dibujar (el decoder) que el dibujo se parezca lo más posible a la foto original.

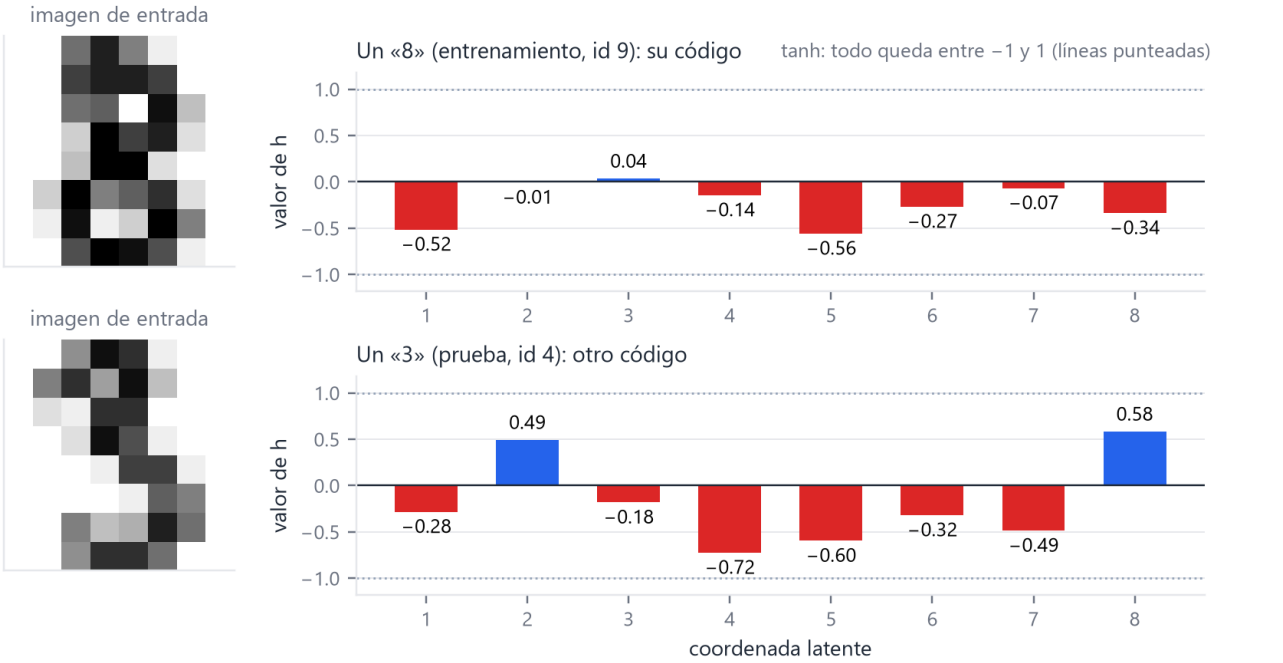
h = f(x) (64 valores → 8 valores)

$\hat{x}$ = g(h) (8 valores → 64 valores)

Con los números de la clase

- El «8» de la diapositiva 9 (id 9, de entrenamiento) tiene h = (−0.52, −0.01, 0.04, −0.14, −0.56, −0.27, −0.07, −0.34)
- Las 8 coordenadas suman −1.86: no son probabilidades y no tienen por qué sumar 1
- Con tanh, todas quedan entre −1 y 1 (en las 1 797 imágenes, entre −0.95 y 0.90)
- Otra imagen produce otros 8 números: los pesos no cambian, lo que cambia es la entrada (diapositiva 9)

Cuidado. Cada coordenada latente **no tiene un significado propio** (no es «el trazo vertical»). Es solo la salida de una función con pesos aprendidos. En PCA sí se ordenan las componentes por importancia; en el vector latente de un autoencoder, no.



Diapositiva 9 (el esquema de arriba es de las diapositivas 3 y 8): las 8 coordenadas latentes de dos imágenes. Rojo = negativo, azul = positivo; las líneas punteadas marcan los límites −1 y 1 de tanh.

2 Reconstruir no es clasificar: la entrada también es la referencia

La mecánica es la misma que en la Clase 45; cambian la salida y la referencia. Un clasificador entrega 10 probabilidades y se compara con la etiqueta; un autoencoder entrega 64 intensidades y se compara con la propia imagen.

1. **Qué es reconstruir.** Estimar de nuevo cada una de las 64 intensidades y colocarla en su mismo lugar. La salida tiene el mismo tamaño y el mismo significado que la entrada (diapositiva 5).
2. **La referencia es la entrada.** x entra al modelo y se conserva para evaluar la salida: la pérdida compara x y $\hat{x}$ píxel a píxel (diapositiva 7). Es como un modelo de regresión donde lo «observado» es la propia entrada.
3. **Sin etiquetas.** La etiqueta del dígito no interviene en la pérdida de reconstrucción. Por eso se puede entrenar con muchas imágenes sin etiquetar (diapositiva 6).
4. **Para qué sirve.** Tres usos de la diapositiva 4: buscar imágenes parecidas comparando vectores latentes; recuperar una señal limpia (se entrena con pares ruido/limpio: es otra tarea); priorizar una revisión cuando el error es alto. El uso decide qué hay que conservar y cómo comprobar que ayuda.

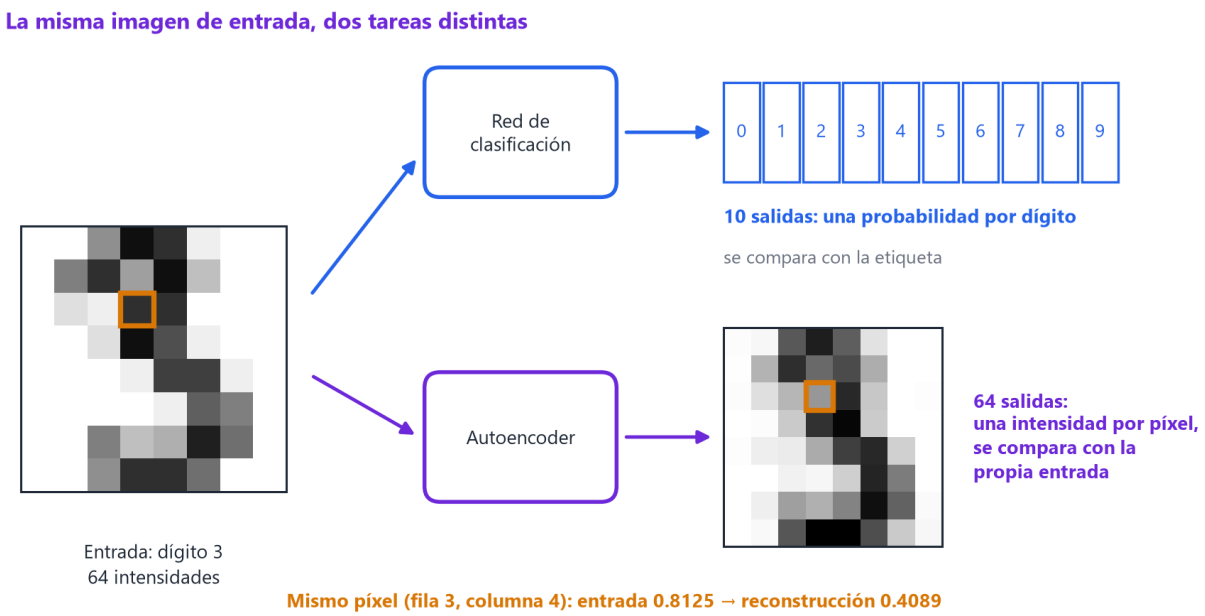
En palabras simples

Antes la red respondía **«¿qué dígito es?»** y se corregía con la etiqueta que puso una persona. Ahora responde **«¿cómo se ve esto?»** y se corrige comparando su dibujo con la imagen original. Es como copiar un dibujo de memoria: nadie tiene que decirte qué dibujo era, basta con poder comparar tu copia con el original.

Con los números de la clase

- Un píxel (fila 3, columna 4) vale 0.8125 en la entrada y la red lo estima en 0.4089 (diapositiva 5)
- Salida de clasificación: 10 valores (uno por dígito); salida de reconstrucción: 64 valores (uno por píxel)
- Misma imagen de entrada, mismas capas intermedias y la misma retropropagación; cambia la tarea final (transcripción)

Cuidado. Reconstruir bien no garantiza que el código sirva para clasificar bien: en la diapositiva 36, dos entradas con etiquetas distintas pueden tener el mismo vector latente. Se comprueba con datos en la sección 7.



Diapositivas 5 y 6: la misma imagen (un 3 de prueba, id 4) con las dos tareas; el cuadro naranja es el mismo píxel en la entrada y en la reconstrucción.

3 Resumir es perder: el cuello de botella y los 8 números reales

Si dos entradas distintas producen el mismo vector latente, el decoder no puede distinguirlas. Lo importante es qué información se pierde, no solo cuánta.

1. **Ejemplo pequeño (diapositiva 11).** Imágenes de 2 píxeles: (0.2, 0.8) y (0.8, 0.2). Un encoder fijo que solo guarda el promedio $h = (x_1 + x_2)/2$ da $h = 0.5$ para ambas, y un decoder fijo que repite h devuelve (0.5, 0.5) para las dos.
2. **Qué se perdió.** El orden: cuál píxel era claro y cuál oscuro. El decoder es determinista: mismo h, misma salida. Ningún entrenamiento posterior puede recuperar lo que el encoder no guardó.
3. **No todo resumen pierde lo mismo.** Con otro encoder (ejemplo propio, $h = x_1 - x_2$) el código también es un solo número, pero conserva el orden de estas dos imágenes. Un buen resumen es el que guarda lo que el uso necesita.
4. **8 números reales ≠ 8 bits.** Cada coordenada es un número real entre −1 y 1, con tantas cifras como haga falta, no un 0 o un 1. Ocho bits solo distinguen $2^8 = 256$ combinaciones. Tampoco equivale a cómo se guardan los datos en el computador (diapositiva 10).

En palabras simples

Si un guardia de seguridad solo anota el **promedio** de las alturas de dos personas, nunca podrá decir cuál era la más alta. El resumen «promedio» descartó justo la diferencia. Los números reales del código son como una regla con milímetros; los bits serían una regla con solo dos marcas.

$h = (x_1 + x_2)/2$ y $\hat{x} = (h, h)$

$\ell = ((0.5 - 0.2)^2 + (0.5 - 0.8)^2) / 2 = 0.09$

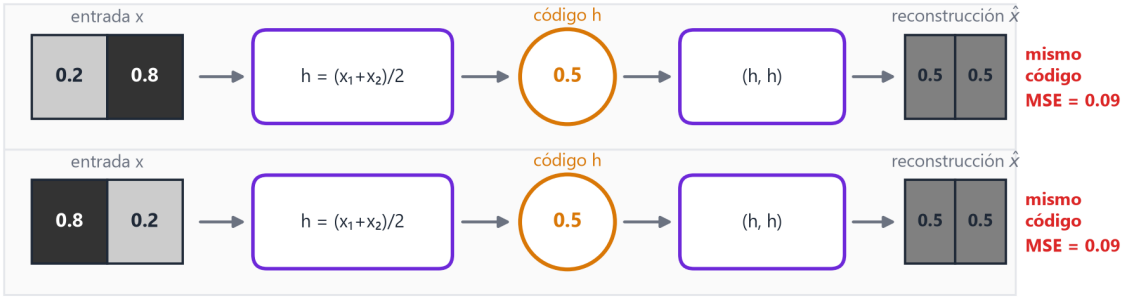
Con los números de la clase

- (0.2, 0.8) → $h = 0.5$ → (0.5, 0.5), MSE 0.09; (0.8, 0.2) → $h = 0.5$ → igual, MSE 0.09
- Ejemplo propio: $h = x_1 - x_2$ da $h = -0.6$ y 0.6: códigos distintos y MSE 0.00 en ambos
- Nuestra reproducción: con el código real el MSE de validación es 0.0193; si se reemplaza por su signo (±1), sube a 0.1277
- Redondear el código a 1 decimal da 0.0197 y a 2 decimales 0.0193: el decoder usa la parte fina de los números
- Con 8 signos solo hay 256 códigos posibles; en las 360 imágenes de validación aparecen 104

Cuidado. El experimento de los signos no demuestra cuántos bits «vale» el código: el decoder se entrenó con valores reales y no se reentrenó para códigos binarios. Solo muestra que los números reales llevan información fina.

Ejemplo pequeño de la clase: 2 píxeles → 1 número → 2 píxeles

Encoder fijo $h = (x_1 + x_2)/2$ y decoder fijo (h, h), como en la diapositiva 11:

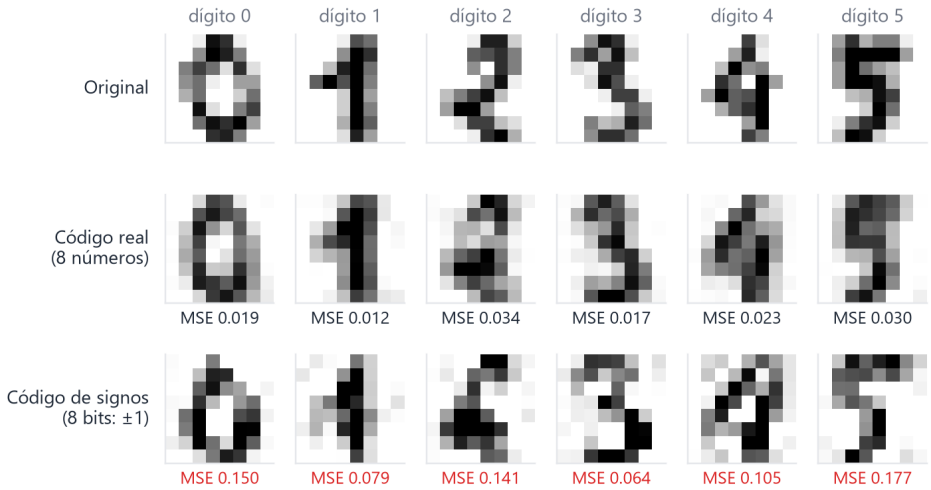


Ejemplo propio (no es de la clase): otro encoder fijo, $h = x_1 - x_2$, y decoder $(0.5 + h/2,\ 0.5 - h/2)$:



Diapositiva 11 (arriba) y ejemplo propio (abajo): el mismo tamaño de código puede perder o conservar el orden. Los colores van de blanco (0) a negro (1).

8 números reales no son 8 bits · MSE de validación: código real 0.0193 frente a solo signos 0.1277



Nuestra reproducción: el decoder entrenado recibe el código real (fila central) o solo su signo ±1 (fila inferior), sin reentrenar. Primer caso de cada dígito 0 a 5 en prueba.

4 El error de reconstrucción: MSE por píxel, a mano

El error de una imagen es el promedio, sobre sus 64 píxeles, de la diferencia al cuadrado entre la reconstrucción y la entrada. El error del conjunto es el promedio de los errores de las imágenes.

1 **Por píxel.** Se resta la entrada a la reconstrucción en cada posición j y se eleva al cuadrado: $(\hat{x}_j - x_j)^2$. Un píxel puede equivocarse hacia más oscuro o hacia más claro: con el cuadrado, −0.3 y +0.3 cuestan lo mismo (0.09).

2 **Por imagen y por conjunto.** $\ell(x, \hat{x}) = (1/64)\ \Sigma_j\ (\hat{x_j} - x_j)^2$. Luego L = promedio de los ℓ de las n imágenes. Es una medida como el error cuadrático medio de una regresión, aplicada pixel a pixel.

3 **Lo que no dice.** El MSE no dice en qué dirección se equivoca el modelo ni en qué píxel; para eso hay que mirar las imágenes y el mapa de error. El cuadrado penaliza más los errores grandes.

4 **Cero no es el objetivo.** Una copia perfecta de la entrada (la identidad) tiene error 0 y no resume nada. Si el vector latente conservara todos los detalles, no habría aprendido nada útil. Hace falta una restricción adecuada y evaluar fuera de los datos de ajuste (diapositiva 14).

En palabras simples

Piensa en una **multa por cada píxel mal dibujado**, que crece con el cuadrado del error: equivocarse por poco casi no cuesta; equivocarse mucho cuesta muy caro. La nota final de la imagen es el promedio de las 64 multas.

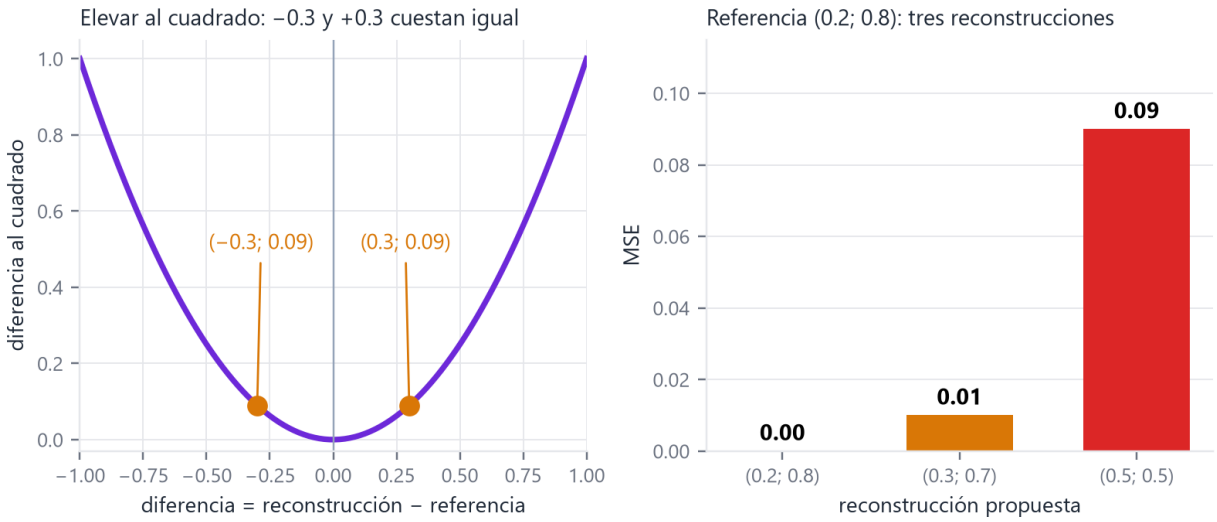
$$\ell(x, \hat{x}) = (1/64)\ \Sigma_j\ (\hat{x_j} - x_j)^2$$

$$L = (1/n)\ \Sigma_i\ \ell(x_i, \hat{x_i})$$

Con los números de la clase

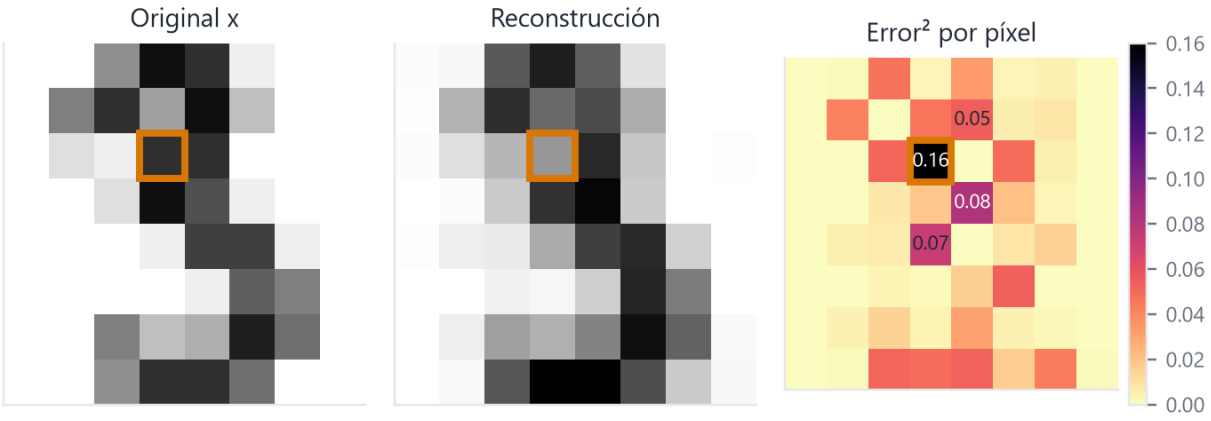
- Referencia (0.2; 0.8) y reconstrucción (0.5; 0.5): ℓ = (0.09 + 0.09)/2 = 0.09; con (0.3; 0.7): 0.01; con (0.2; 0.8): 0.00
- Píxel de la diapositiva 5: $(0.4089 - 0.8125)^2 = (-0.4036)^2$ = 0.1629 (diapositiva 29: ≈ 0.1629)
- Los 64 cuadrados suman 1.1021; promediados dan el MSE de esa imagen: 0.017220 (diapositiva 29: 0.017220)
- Ese solo píxel concentra el 14.8 % de la suma de errores de la imagen

Cuidado. El MSE depende de la escala: aquí las intensidades van de 0 a 1 (se dividen por 16). Un «0.017» solo se entiende al compararlo con una referencia.



Diapositivas 12 y 13: la diferencia al cuadrado y las tres reconstrucciones propuestas para la referencia (0.2; 0.8).

Dígito 3 (id 4): MSE de la imagen = promedio de los 64 cuadrados = 0.017220



Cuadro naranja (fila 3, columna 4): $(0.4089 - 0.8125)^2 = (-0.4036)^2 = 0.1629$

Diapositiva 29: el píxel de la diapositiva 5 aporta el mayor cuadrado de la imagen; el MSE de la imagen es el promedio de los 64 cuadrados.

5 Referencias para comparar: imagen media y PCA con 8 componentes

Un MSE aislado no dice si es bueno: hay que compararlo con referencias simples. En validación: imagen media 0.0729, PCA con 8 componentes 0.0235 y el autoencoder profundo 0.0193.

1. **Imagen media.** Ignora la entrada y devuelve siempre el promedio de las imágenes de entrenamiento. Es el piso de «no saber nada de esta imagen».
2. **PCA con 8 componentes.** Proyecta cada imagen sobre 8 direcciones ($z = (x - \bar{x})P_k$) y reconstruye en ese subespacio ($\hat{x} = \bar{x} + zP_k^T$). Es una reconstrucción **lineal** con un resumen también de 8 números, ajustada solo con entrenamiento.
3. **Autoencoder.** Reconstrucción **no lineal**: las capas ocultas con tanh permiten formas que una proyección lineal no captura. Se compara en las mismas imágenes y con la misma escala (diapositiva 27).
4. **Cómo leer la comparación.** Menor MSE es mejor. Pero esto responde solo una pregunta (¿reconstruye?): aún falta evaluar la utilidad del código para otra tarea.

En palabras simples

La imagen media es el **retrato robot** que sirve para todos y no se parece a nadie. PCA es un dibujante que solo sabe usar **8 plantillas** combinadas con regla y compás. El autoencoder es un dibujante que puede combinar los 8 números de forma más flexible (curvas); por eso, si lo entrenamos bien, puede acercarse más al original.

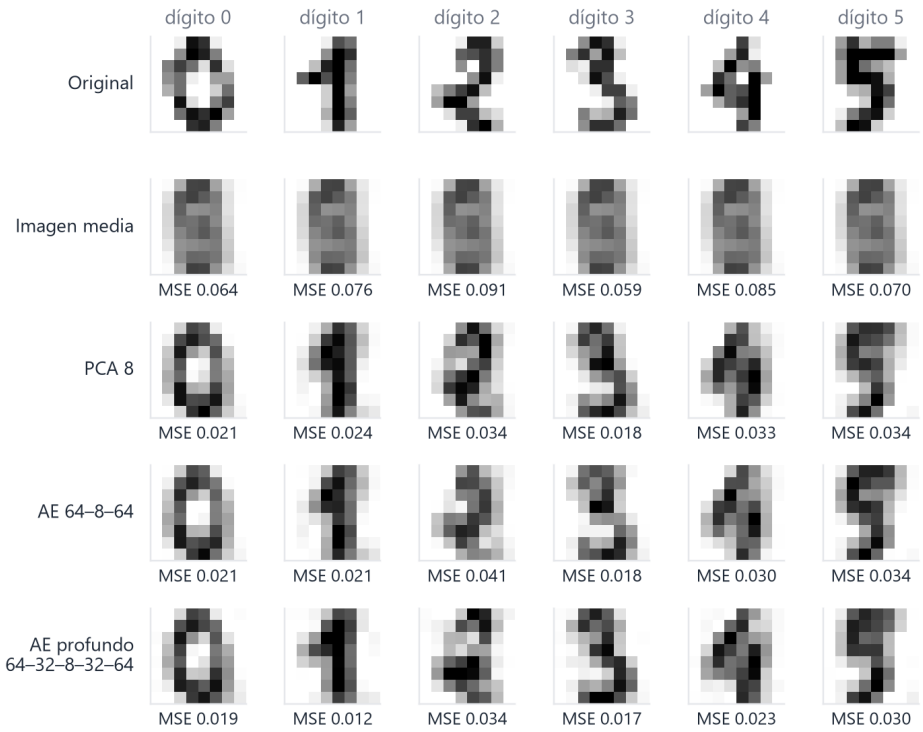
$$z = (x - \bar{x})\,P_k$$

$$\hat{x} = \bar{x} + z\,P_k^T$$

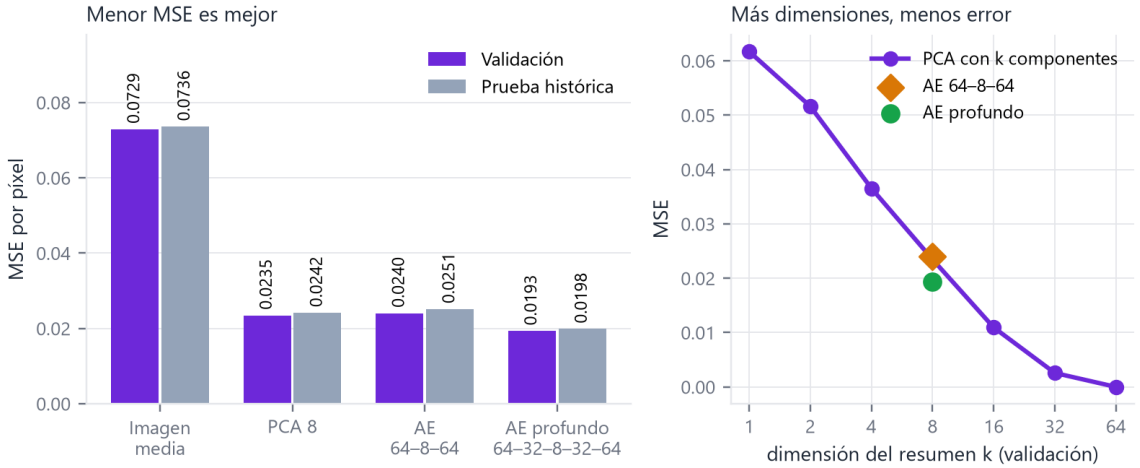
Con los números de la clase

- Imagen media: validación 0.0729 y prueba 0.0736; PCA 8: 0.0235 y 0.0242; autoencoder elegido: 0.0193 y 0.0198 (diapositiva 27)
- PCA 8 reduce el error de la media en 67.8 %; el autoencoder profundo, en 73.5 % y respecto de PCA 8 en 17.7 % (nuestro cálculo)
- Nuestra reproducción del 64–8–64: 0.0240 en validación; casi igual a PCA 8 (0.0235)
- PCA con k = 1, 2, 4, 8, 16, 32 y 64: 0.0617, 0.0516, 0.0366, 0.0235, 0.0109, 0.0026 y 0 con k = 64 (se usa toda la información)

Cuidado. Una semilla y una corrida no establecen estabilidad, y la prueba ya se mostró en clase: la comparación es retrospectiva. Que gane el autoencoder en esta corrida no prueba superioridad en general (diapositiva 47).



Diapositiva 28: primer caso de cada dígito 0 a 5 en prueba. Solo la visualización recorta las intensidades a [0, 1]; el MSE de cada imagen usa las salidas completas.



Diapositivas 27 y 30: MSE de las cuatro reconstrucciones en validación y prueba (izquierda) y PCA con distintos k (derecha; nuestra reproducción).

6 Profundidad, parámetros y entrenamiento

Se compararon dos arquitecturas con el mismo vector latente de 8 valores: 64–8–64 (1 096 parámetros) y 64–32–8–32–64 (4 744 parámetros). Se elige con el MSE de validación.

1. **Receta.** Capas ocultas con tanh y salida lineal; Adam con tasa 0.002; lotes de 64 imágenes; penalización de pesos alpha = 0.0001; hasta 150 épocas por candidata; referencias: imagen media y PCA 8 (diapositiva 23).
2. **Latente y parámetros no son lo mismo.** Ambas redes resumen cada imagen en 8 valores, pero la que tiene capas adicionales de 32 unidades aprende más pesos y sesgos. Cada capa aporta unidades × (entradas + 1) parámetros.

3 **Cómo aprende.** 1) reconstruir $x \to h \to \hat{x}$; 2) medir $\ell(x, \hat{x})$; 3) retropropagar los gradientes hacia las dos mitades; 4) actualizar pesos y sesgos; 5) repetir con el lote siguiente; al terminar cada época se mide la validación *sin* actualizar.

4 **El límite de épocas.** Los mínimos de validación están en la última época (la 150): el presupuesto de entrenamiento limita la conclusión, porque la curva aún bajaba (diapositiva 26).

En palabras simples

Una capa más es como añadir **una estación intermedia en una cadena de montaje**: la imagen pasa por más transformaciones antes de llegar al resumen de 8 números. Más estaciones dan más flexibilidad, pero también más piezas que ajustar y más tiempo; y que la curva siga bajando al final significa que no sabemos si ya se llegó al fondo.

parámetros de una capa = unidades × (entradas + 1)

64–8–64: 8·(64+1) + 64·(8+1) = 520 + 576 = 1 096

Con los números de la clase

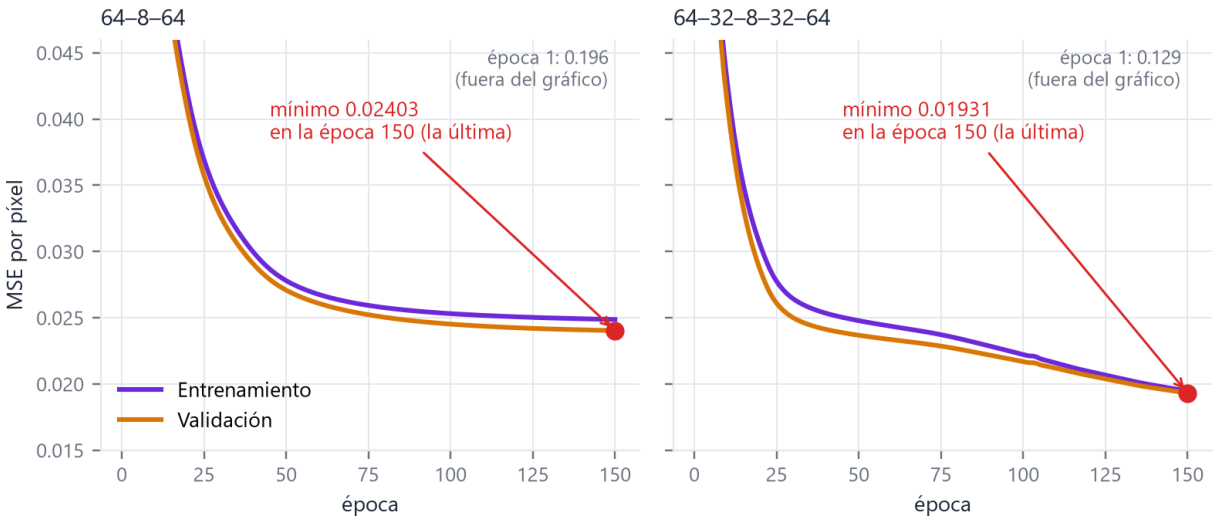
- 64–8–64: 520 + 576 = 1 096; 64–32–8–32–64: 2 080 + 264 + 288 + 2 112 = 4 744 (diapositiva 18)
- MSE de validación al final de la época 150: 0.02403 (64–8–64) y 0.01931 (64–32–8–32–64); en la época 1: 0.1960 y 0.1287
- Entrenamiento y validación casi coinciden (época 150: 0.02487 y 0.01945 en entrenamiento): no se ve sobreajuste en este presupuesto
- Protocolo (diapositiva 22): 1 077 imágenes de entrenamiento, 360 de validación y 360 de prueba histórica; PCA y redes se ajustan con entrenamiento

Cuidado. Elegir la arquitectura y la época con los resultados de *prueba* también es fuga de información. La elección se hace con validación; la prueba queda para evaluar la receta ya fijada (diapositiva 21).



Sobre cada flecha: pesos + sesgos de esa capa (unidades × (entradas + 1)). Ambas redes resumen cada imagen en 8 valores; la profunda tiene más parámetros.

Diapositivas 17 y 18: tamaños por capa y parámetros (pesos + sesgos) de cada capa, calculados con unidades × (entradas + 1).



Diapositivas 25 y 26: MSE por píxel (sin penalización) en cada época, con la mejor época marcada en rojo.

7 Mirar el código y usarlo para otra tarea

El vector latente puede servir para otras tareas, pero cada uso necesita su propia evaluación. Un buen MSE de reconstrucción no prueba que el código ayude a clasificar, buscar o alertar.

1. **Mirar 2 de las 8 coordenadas.** Se pueden graficar dos coordenadas y colorear por dígito; los colores son etiquetas añadidas solo para inspeccionar, el autoencoder no las usó (diapositiva 32). Dos de ocho no cuentan toda la historia.
2. **Buscar parecidos.** Se comparan los 8 valores del vector latente y se ordenan las imágenes por cercanía (diapositiva 48). Útil si «parecido» en el código coincide con «parecido» para la tarea.
3. **Clasificar con representaciones fijas.** Se congelan PCA y el autoencoder, se transforman las imágenes y se ajusta una regresión logística con estandarización y C = 1 sobre entrenamiento; se compara con la misma receta sobre píxeles (diapositivas 34 y 35).
4. **Qué evaluar según el uso.** Reconstruir: error sobre la entrada. Clasificar: métricas con etiquetas. Buscar vecinos: relevancia de lo recuperado. Alertar: errores y utilidad en condiciones reales (diapositiva 33).

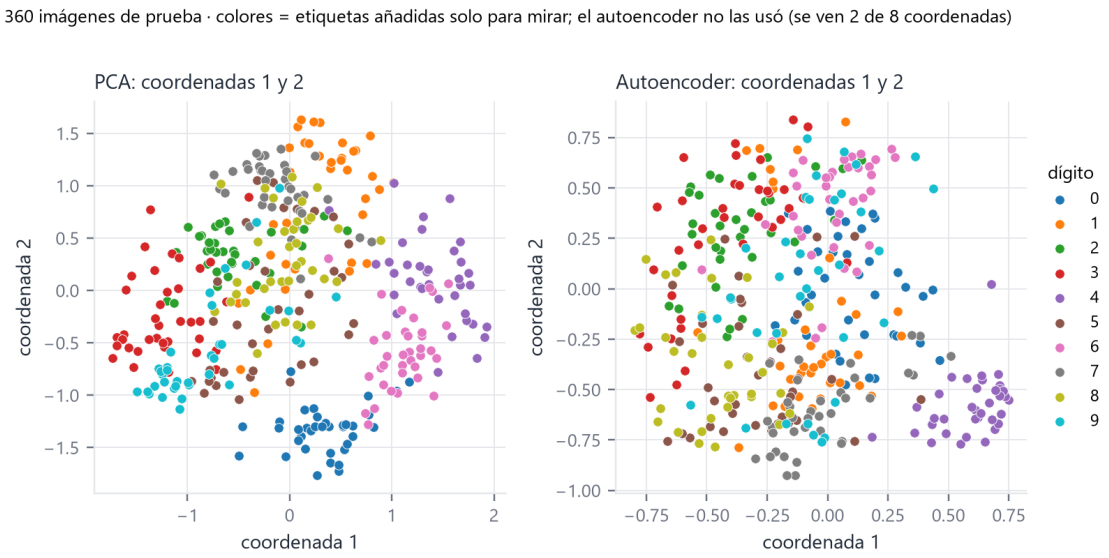
En palabras simples

Es como resumir un libro en 8 palabras clave. Para **encontrar libros parecidos** quizá basta; para **decidir de qué género es** quizá faltan palabras; y para **detectar una errata** no sirve. Cada uso tiene su propia prueba.

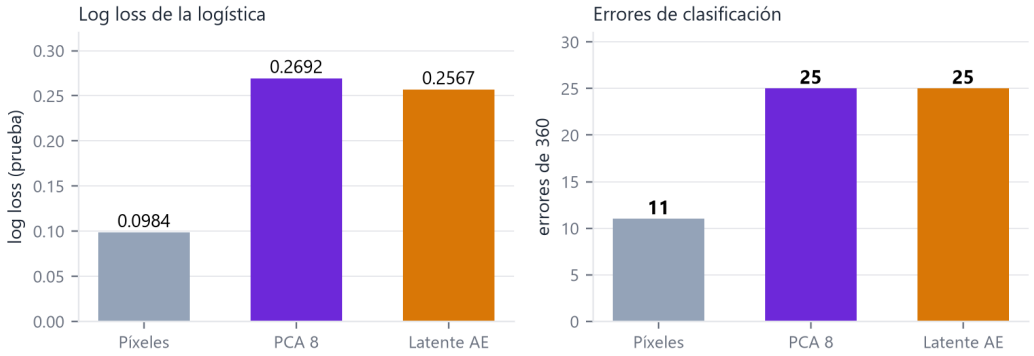
Con los números de la clase

- Vecinos del 8 (id 29) en entrenamiento: distancias 0.246, 0.284, 0.311, los tres son 8 (diapositiva 48)
- Logística sobre píxeles: log loss 0.0984 y 11 errores de 360; sobre PCA 8: 0.2692 y 25; sobre el latente del AE: 0.2567 y 25 (diapositiva 35)
- Los dos vectores de 8 valores dan 25 errores; los píxeles, 11: **la utilidad depende de la receta**
- Diapositiva 36: en un ejemplo artificial, dos entradas de etiquetas distintas se reconstruyen con MSE 0.005 y tienen el mismo vector latente

Cuidado. Que PCA y el autoencoder den 25 errores en esta receta no demuestra que ninguno sirva para clasificar con otro clasificador, ni que los píxeles sean siempre mejores.



Diapositiva 32: coordenadas 1 y 2 de PCA y del autoencoder para las 360 imágenes de prueba, coloreadas por dígito.



Diapositiva 35: logística con píxeles, con PCA 8 y con el vector latente del autoencoder; prueba histórica de 360 imágenes.

8 El error de reconstrucción como alerta de anomalías, y sus límites

Si el modelo aprendió a reconstruir lo habitual, un error alto sugiere que la imagen no se parece a lo habitual. El puntaje e(x) es el MSE de la imagen y se activa una alerta si e(x) > c; no es una probabilidad de anomalía.

1. **Puntaje y umbral.** $e(x) = (1/64)\ \Sigma_j\ (\hat{x}_j - x_j)^2$. El umbral c se fija antes de mirar la prueba: aquí es el percentil 95 de los 360 errores de validación (diapositivas 38 y 39).
2. **Qué significa el percentil.** Por construcción, el 5 % de la validación queda sobre el corte. No es un 95 % de confianza sobre defectos: el percentil de un puntaje no es un nivel de confianza (diapositiva 46).
3. **Prueba de estrés artificial.** Se desplaza cada imagen una columna a la derecha y se compara cada salida con *su propia* entrada desplazada. Muestra sensibilidad a un cambio, no detección validada de defectos reales (diapositivas 40 y 41).
4. **Cuándo falla una alerta.** Variación habitual no representada (muchos avisos sobre casos aceptables); anomalías en el entrenamiento y un decoder flexible (aprende a reconstruirlas); cambio de adquisición (el puntaje refleja el cambio técnico). Se necesita una validación específica (diapositiva 42).

En palabras simples

Es como el **detector de humo** de una cocina: suena cuando el ambiente se aleja de lo habitual, pero puede sonar por una tostada quemada (falsa alarma) y no suena si el humo es «habitual» para él. El sonido no te dice si hay fuego, solo que hay que ir a mirar.

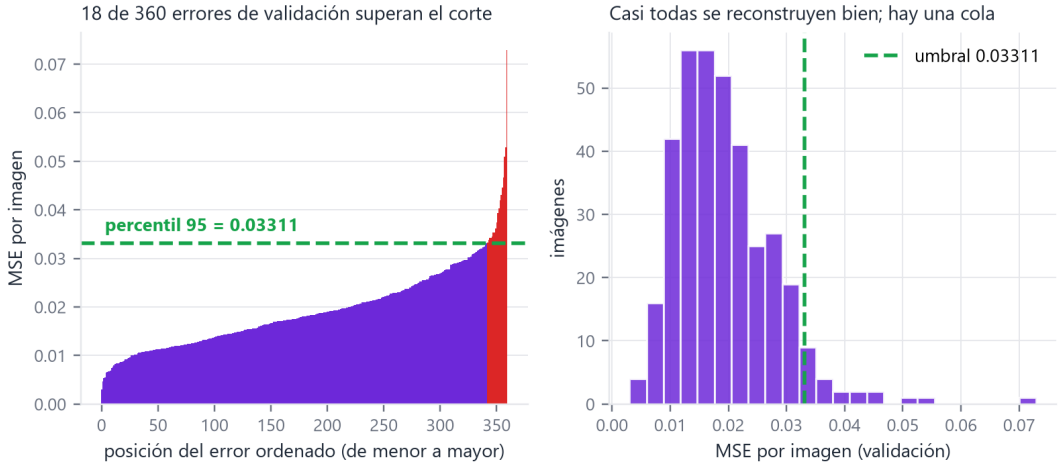
$$e(x) = (1/64)\ \Sigma_j\ (\hat{x_j} - x_j)^2$$

alerta si $e(x) > c$

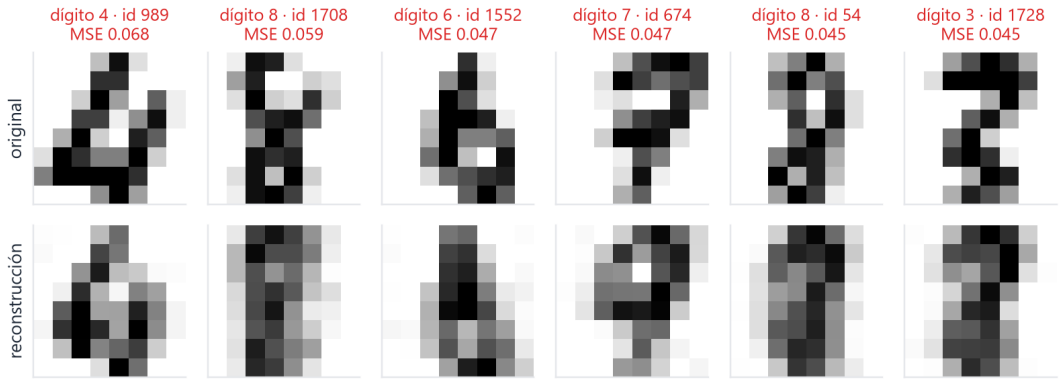
Con los números de la clase

- Umbral (percentil 95 de validación): c = 0.033108; 18 de 360 errores de validación lo superan (5 %)
- En prueba, originales: 35 de 360 alertas (9.72 %); desplazadas: 319 de 360 (88.61 %) (diapositiva 40)
- MSE de la imagen 3 (id 4): 0.0172; desplazada: 0.0418 (diapositiva 41)
- Caso de la diapositiva 45: «rechazar el 5 % con mayor error» no equivale a «detectar cualquier defecto con 95 % de confianza»

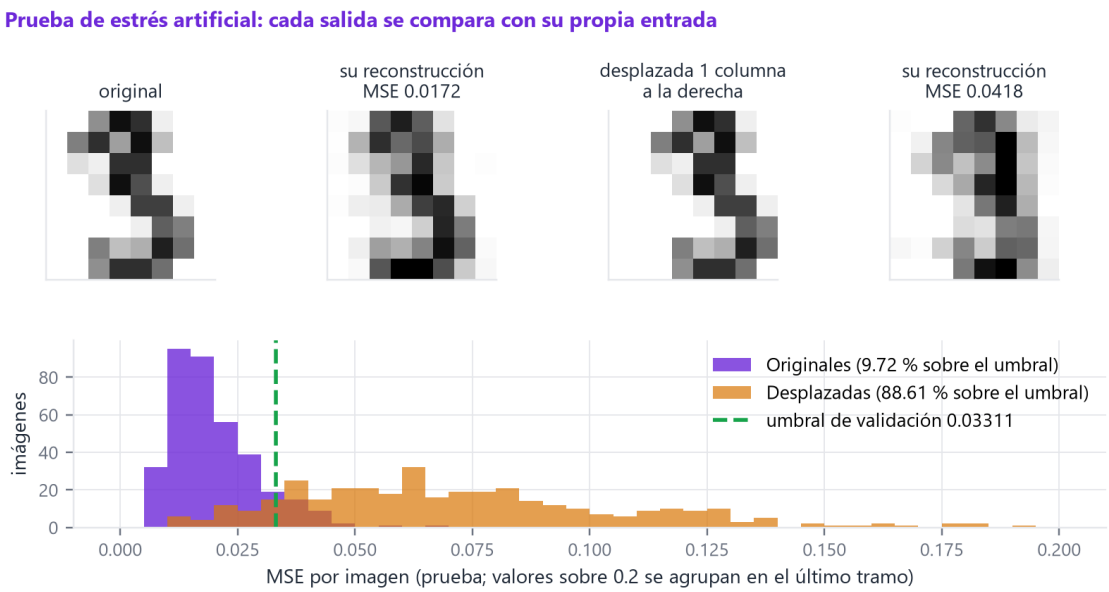
Cuidado. Selección y umbral usaron la misma validación: no garantizan falsas alarmas futuras. Y el estrés artificial no mide sensibilidad ni especificidad frente a anomalías reales.



Diapositivas 38 y 39: errores de validación ordenados (izquierda, en rojo los que superan el corte) e histograma (derecha).



Las seis imágenes de prueba con mayor error de reconstrucción (nuestra reproducción): todas superan el umbral; una alerta no confirma un defecto.



Diapositivas 40 y 41: arriba, la imagen 3 original y desplazada con sus reconstrucciones; abajo, los errores de las 360 imágenes de prueba originales y desplazadas.

Repaso, alertas del material y criterio de dominio

Lo que debes poder explicar sin apuntes

Alertas del material

Qué conviene saber al leer las diapositivas y la transcripción

- **La transcripción tenía errores de reconocimiento de voz** («DeepLens» por deep learning; «Ableton Core» y «topcoder» por autoencoder; «tarjeta hiperbólica» por tangente hiperbólica; «RER meronales» por redes neuronales; «rich o lazo» por ridge o lasso; «0208» por 0.2, 0.8; «peso y se asco» por pesos y sesgos). Se corrigieron en 49 líneas; el original se conserva en `VideosClases/originales_sin_corregir/`.
- **La grabación termina en el recreo** (1:14:23): no incluye las diapositivas 15 a 53. Lo que esas láminas dicen se explica aquí desde las láminas y el programa, no desde lo hablado.
- **«Pérdida logarítmica» en la clase.** Al describir el ciclo de entrenamiento se habló de una pérdida logarítmica; la pérdida de reconstrucción de las diapositivas y del programa es el **MSE** (log loss solo aparece al clasificar, diapositiva 35).
- **«Clasificar de manera no supervisada».** Se sugirió oralmente que el autoencoder permitiría clasificar sin etiquetas; las diapositivas 35 y 36 muestran que un buen código de reconstrucción no garantiza una buena clasificación (25 errores frente a 11 con píxeles).
- **Resumir no es reducir el archivo.** En clase se aclaró que pasar de 64 a 8 no busca achicar la imagen: las 8 coordenadas controlan cuántos números la describen; no es compresión de archivo ni equivale a 8 bits.
- **El código de la diapositiva 9** corresponde a un «8» de entrenamiento (id 9), y el 3 de las diapositivas 5, 29 y 41 es el id 4 de prueba.
- **Las cifras de R son otra corrida.** Las diapositivas usan la ejecución de Python; R entrena con RSNNS y descenso de gradiente y puede diferir.

Preguntas de repaso (intenta responder antes de leer)

#	Pregunta	Respuesta corta
1	Encoder $h = (x_1 + x_2)/2$ y decoder (h, h). Para $x = (0.2, 0.8)$, ¿qué h y qué reconstrucción? ¿Y el MSE?	$h = 0.5$; $\hat{x} = (0.5, 0.5)$; MSE $= (0.3^2 + 0.3^2)/2 = 0.09$.
2	¿Cuántos valores tienen x, h y $\hat{x}$ en 64–8–64? ¿Cuántos parámetros?	64, 8 y 64; 520 + 576 = 1 096.
3	Referencia (0.2, 0.8) y reconstrucción (0.3, 0.7). ¿Cuánto vale ℓ?	$((0.1)^2 + (0.1)^2)/2 = 0.01$.
4	¿Por qué un error de reconstrucción 0 no es la meta?	La identidad lo logra sin resumir nada; hace falta un cuello de botella y evaluar fuera del ajuste.
5	Un píxel vale 0.8125 y se reconstruye como 0.4089. ¿Cuánto aporta al MSE de la imagen?	$(-0.4036)^2 = 0.1629$, que se promedia con los otros 63 cuadrados.
6	¿Con qué dos referencias se compara el autoencoder y qué valores dan en validación?	Imagen media 0.0729 y PCA 8 0.0235; el autoencoder elegido 0.0193.
7	¿Con qué conjunto se elige entre las dos arquitecturas y qué limita la conclusión?	Con validación (MSE); ambos mínimos están en la época 150, así que el presupuesto de épocas limita la conclusión.
8	Un puntaje de reconstrucción de 0.05, ¿es una probabilidad de anomalía?	No: es un MSE; solo se compara con un umbral que debe validarse para el uso previsto.
9	Umbral = percentil 95 de validación: ¿qué porcentaje de validación lo supera y por qué en prueba salió 9.72 %?	5 % por construcción (18 de 360); en prueba (35 de 360) la proporción puede diferir porque el umbral se fijó con otras imágenes.

Criterio de dominio

- ☐ Explico con mis palabras qué es una representación, un encoder, un decoder y un vector latente
- ☐ Calculo a mano la reconstrucción de un autoencoder de juguete y su MSE por píxel y por imagen
- ☐ Cuento los parámetros de 64–8–64 y de 64–32–8–32–64, y explico por qué latente y parámetros no son lo mismo
- ☐ Explico por qué una copia perfecta no es la meta y por qué 8 números reales no equivalen a 8 bits
- ☐ Comparo un autoencoder con la imagen media y PCA, y digo qué limita esa conclusión
- ☐ Explico por qué un puntaje de reconstrucción no es una probabilidad y qué validación falta para usarlo como alerta

Cifras de las diapositivas frente a nuestra reproducción (programa del profesor)

Cifra	Diapositivas	Reproducido	¿Igual?
MSE validación 64-8-64 (mejor)	0.02403 · época 150	0.02403 · época 150	sí
MSE validación 64-32-8-32-64 (mejor)	0.01931 · época 150	0.01931 · época 150	sí
Imagen media: validación / prueba	0.0729 / 0.0736	0.0729 / 0.0736	sí
PCA 8: validación / prueba	0.0235 / 0.0242	0.0235 / 0.0242	sí
AE elegido: validación / prueba	0.0193 / 0.0198	0.0193 / 0.0198	sí
Parámetros 64-8-64	520+576 = 1096	520+576 = 1096	sí
Parámetros 64-32-8-32-64	2080+264+288+2112 = 4744	2080+264+288+2112 = 4744	sí
Píxel fila 3, col. 4 (entrada → reconstrucción; cuadrado)	0.8125 → 0.4089; 0.1629	0.8125 → 0.4089; 0.1629	sí
MSE de esa imagen (3, id 4)	0.017220	0.017220	sí
Código del 8 (diap. 9), valores ordenados	−0.56 −0.52 −0.34 −0.27 −0.14 −0.07 −0.01 0.04	−0.56 −0.52 −0.34 −0.27 −0.14 −0.07 −0.01 0.04	sí
Umbral = percentil 95 de validación	0.033108	0.033108	sí
Validación sobre el umbral	18 de 360	18 de 360	sí
Alertas en prueba: original / desplazada	9.72 % / 88.61 %	9.72 % / 88.61 %	sí
MSE del 3 desplazado (diap. 41)	0.0418	0.0418	sí
Clasificación log loss: píxeles / PCA 8 / AE	0.0984 / 0.2692 / 0.2567	0.0984 / 0.2692 / 0.2567	sí
Clasificación: errores de 360	11 / 25 / 25	11 / 25 / 25	sí
Vecinos del 8 (id 29): distancias	0.246 · 0.284 · 0.311	0.246 · 0.284 · 0.311	sí
Ejemplo pequeño, referencia (0.2, 0.8): MSE con (0.5, 0.5) y con (0.3, 0.7)	0.09 y 0.01	0.09 y 0.01	sí

No reproducible con el programa: Diapositiva 20 (MSE baja de 0.0400 a 0.0298): el programa no incluye ese paso aislado; Diapositivas 12 y 36: ejemplos conceptuales sin código.